

Re-conceptualizing requirements engineering: Findings from a large-scale, agile project

Tor Erlend Fægri
SINTEF ICT
Strindveien 4
7465 Trondheim
+47 73 59 30 00
Tor.E.Fegri@sintef.no

Nils Brede Moe
SINTEF ICT
Strindveien 4
7465 Trondheim
+47 73 59 30 00
Nilsm@Sintef.no

ABSTRACT

Agile development is a force that reshapes requirements engineering (RE). Agile RE processes are not constrained to pre-development phase like in traditional RE; they're evenly spread throughout development. High-level RE occurs at the project beginning, and then agile RE continues at each development cycle. Each development cycle starts with customer representatives discussing requirements with the development teams. To seek new understanding of the requirements engineering process in agile projects, we have applied the model of software development as a conversation. In this paper we describe this model and use it to explain a selection of findings from a case study in a large-scale agile project.

Categories and Subject Descriptors

D.2.1 [Software engineering]: Requirements/specifications.

General Terms

Management, Human Factors.

Keywords

Requirements engineering, software process, agile methods.

1. INTRODUCTION

Software development can be understood as an ongoing, somewhat chaotic and unpredictable conversation among a number of people. In this conversation, ideas concerning objectives, requirements and proposals for designs are subjected to discussions, confrontations and agreements [1]. Gradually, the conversation produces and affirms a multitude of design decisions that are finally 'crystallized' into software code [2, 3].

This conversation model of software development illustrates the blurred boundaries between requirements engineering and software implementation as it unfolds in practice. In fact, requirements engineering is deeply intertwined with a multitude of processes within the software organization [4].

Yet, it is a natural tendency in complex human endeavors to seek control through separation of concerns. This is the fundamental idea that motivates departmentalization, stage-gate processes, the dividing of work into pre-determined phases, and so on. In software development this is evident in the attempt to separate problem-related activities (i.e. requirements engineering) from solution-related activities (i.e. software design, implementation, etc.). Requirements engineering, in its traditional form, is predominantly concerned with the problem space, thus creating a potentially troublesome gap between the problem- and solution space [5]. Although the movement between the problem-world and the solution-world can seem both unpredictable and uncontrollable [6] we should device processes and practices that embrace this somewhat chaotic reality of requirements engineering.

Agile development meets this challenge head on. Agile methods embrace the notion of emergent designs where the building of understanding of the problem is interleaved with the building of working design elements [7]. Consequently, agile development triggers a reshaping of requirements engineering [8] by relaxing the ordering of requirements engineering activities [9] and allowing requirements engineering activities to run in parallel with software implementation. Now, requirements engineering becomes a more continuous, more collaborative and more feedback-driven process.

However, to our knowledge, the conversation model of software development has not yet been applied in research on requirements engineering in agile development. This is an interesting approach because the conversation model sets focus on the knowledge-creating processes that permeates all activities in software development. In particular, improving the shared understanding of requirements is a key enabler for increased productivity in software projects as it has positive knock-on effects on a multitude of other software processes [4]. More importantly, the merging of 'deep knowledge' from both problem and solution domains in cooperative relationships is crucial for building good, useful systems [10]. This form of collaborative learning is a core pillar of agile software development [7] and effectively enables the conversation between spirals of designs, experience and feedback that produces the final software system.

So, what's in a conversation? A conversation may consist of a mixture of different elements, such as the expression of individual ideas, evaluations, examples, references, and suggestions – all stemming from individual's cognitive capacities. Sharing them in the open, making them explicit, enabling their transformation to other, also non-shared forms and thus fuelling the 'generative dance' between knowledge forms allows for productive, knowledge-intensive work [11]. The conversation drives the

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

XP 2015 Workshops, May 25 - 29, 2015, Helsinki, Finland
© 2015 ACM. ISBN 978-1-4503-3409-9/15/05...\$15.00
DOI: <http://dx.doi.org/10.1145/2764979.2764983>

creation of new knowledge, the key ingredient in software development [3]. The conversation model of software development is valuable because it allows us to reason about the aspects of software practice that affects knowledge creation.

Better understanding of requirements engineering in agile development is necessary. Agile methods, as such, are no panacea for requirements engineering. Agile methods cannot alleviate software projects from having to make difficult decisions and dealing with complex tradeoffs in their requirements processes [9]. What agile methods do is to bring these discussions to the foreground of practice, and provide stakeholders with a continuous awareness of how requirements are handled and the consequences thereof – all within an ongoing, collaborative

Norway to employ agile methods. Omega was owned by Gamma, and engaged contractors Alfa and Beta – in addition to resources from Omega itself.

Omega employed roughly 170 people full-time for the bulk of the project's duration. Access to customer representatives and customer participation is critical for agile projects. From the start, Omega persuaded project owners to invest heavily in access to customer personnel. Of the 170 members, 70 were full-time Gamma staff were paid by Omega. Project management ensured that all project members were co-located in the same office space. Co-location was considered an important enabler for effective communication within the project.

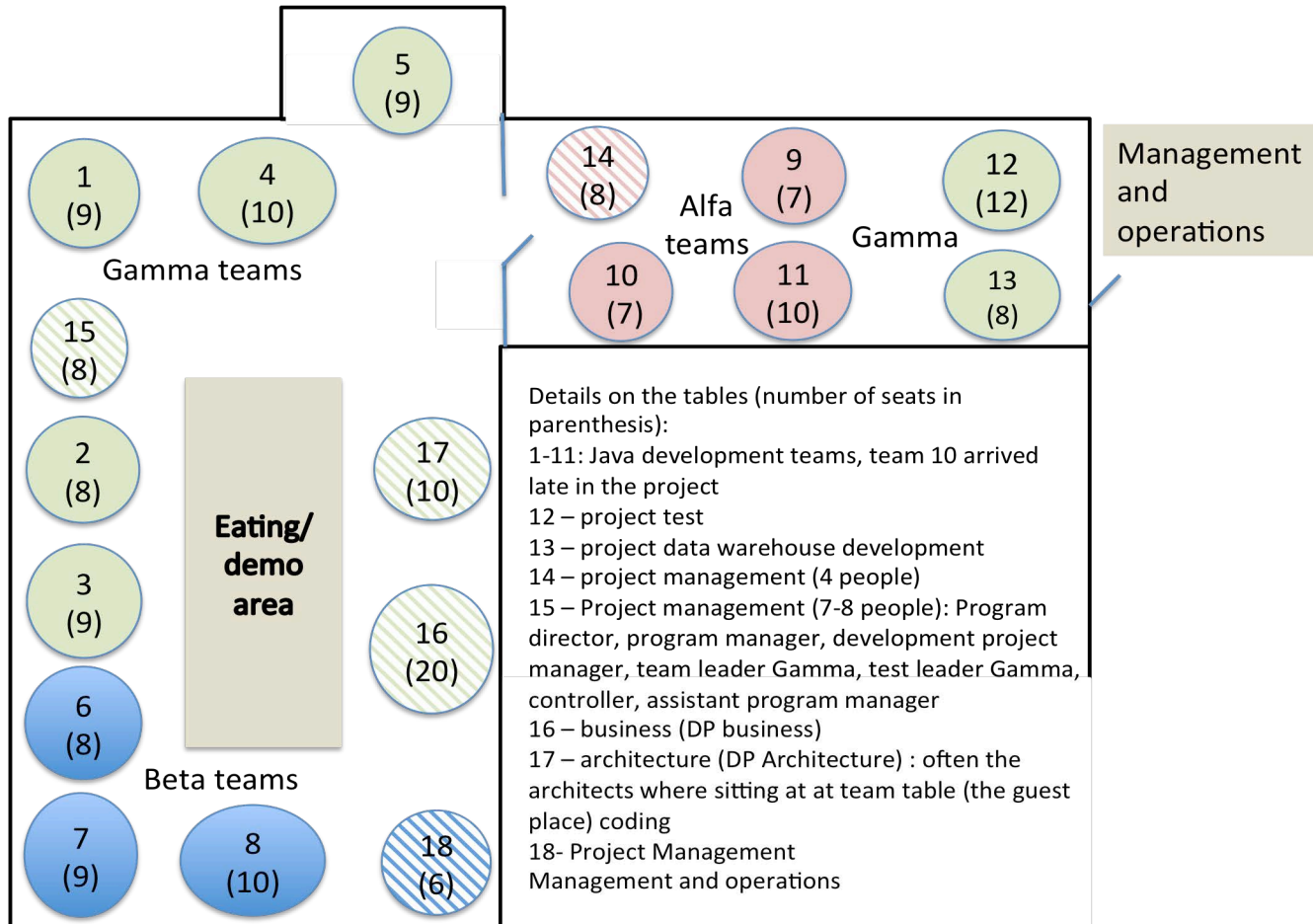


Figure 1. The workspace layout

conversation. Using this view, seeing software development as an ongoing conversation, we will describe how a large-scale agile software project reconciled tensions between the problem- and solution-world. We also illustrate how the conversation model gives us new understanding about how challenges of requirements engineering in agile development can be addressed.

2. THE CASE STUDY

Omega was a project that sought to renew the IT-platform for Gamma, a public body in Norway. Omega run for four years (2008-11) and consumed roughly 800 thousand man-hours. In 2012 it was referred to as one of the largest software projects in

2.1 Data collection

We chose to use two types of data for this case study:

- 1) Project documentation including an official report after project completion (2012), an internal experience report (2012) and a report made to quality assure the project, which was written by an external consulting company in 2010. In total, these reports contained 277 pages of text.
- 2) Focus groups organized by the researchers focusing on four topics of interest (project management, solution descriptions, software architecture, inter-team coordination and knowledge sharing.) For each topic, we organized three focus groups, one for

each organization. We wanted to capture potential differences between the organizations and therefore decided to conduct separate meetings. We asked the three organizations to invite the most relevant people to attend each focus group. In total, we had 33 people participating in focus groups. At *Beta* and *Gamma*, some participated in several groups, making it a total of 26 individuals participating. These people had a number of roles in Omega, from project director, project managers, subproject managers, technical architects, functional architects, tester, development leader, GUI responsible, controller, responsible for information security, scrum master and developers. Note that there is a bias in participation towards employees in management positions; given the availability of people and the topics in the focus groups we did not include any dedicated developers. However, many of the people we talked to had started in the project as developers and gotten other roles subsequently.

To conduct focus group meetings, six researchers formed four teams so that two researchers would participate in each meeting. We developed interview guides, and a rough timeline for the project, which was shown to participants in order to remember key events in the project. The researchers had roles of leading and moderating the discussion and taking notes. All focus groups would start by participants explaining their role in the project. Two focus groups included only one person and were conducted as an interview. One focus group included six persons, and there we made use of brainstorming techniques in order to ensure contribution from everyone invited, to brainstorm on key events and on what they thought was conducted well in the project within the topic of discussion, and what they thought could be improved. All meetings were recorded and transcribed, and we took pictures of drawings on whiteboards. This produced a total of 326 pages of transcribed material. Minutes of the meetings were sent to participants for information and for comments.

2.2 Data analysis

Our findings and discussions are based on qualitative data analysis. The analysis started with the research teams presenting first impressions immediately after data collection. We imported all data documents into a tool for qualitative analysis, and did a descriptive and holistic coding [12] of the core topics. Three researchers read the relevant documents and did individual coding, and then we agreed on coding in joint meetings. Units of text ranged from sentences to whole pages and were coded into code-nodes such as “Practices for solution description” and “Participants in solution description.”

After coding the material, we discussed what were the most interesting findings in the material. This was then made into a slide presentation, and feedback meetings with all three organizations were conducted for member checking. These meetings were recorded and partially transcribed to add further data material.

3. FINDINGS

Generally, conversations were a core vehicle for coordinating work in Omega. Verbal and direct communication was encouraged while written and indirect communication was discouraged. Demonstrating the inspiration to exploit agile principles to the fullest, one example is the Omega project manager for development who proclaimed: “We will not write down our discussions or decisions from meetings. If you need information then you are invited to participate in our meetings.”

As much as possible of the communication in Omega was verbal. This demanded a large number of planned, periodic meetings (demos, project leader meetings, status meetings, Scrum of Scrum meetings (for each contractor) as well as daily standup meetings and MetaScrum.

In addition, there was a plethora of informal meetings. We learned that our respondents felt that the informal meetings were more important than the planned, formal meetings. The development manager for Alfa said: “What is the critical part in agile? It is to deliver something in the next three weeks. Thus, you don't have time to wait for a guy to read all his other e-mail before responding to yours. You need to go over there and ask for attention. The answer will be there in 10 seconds. There is a balance of course, you cannot interrupt the guy too much either.” We think this last point is one of the key elements of why Omega succeeded in implementing a mainly verbal communication scheme. People need to apply their own judgment in when they need to seek information and apply judgment regarding how to make the knowledge transfer as effective as possible. Our data revealed that it took some time before people found sustainable balances between participating in meetings, talking to other people and working solitary. Furthermore, participants learned who to talk to and this form of pragmatic networking was used extensively to seek information and clarifications. Having a large network where coordination and communication was based on face-to-face meetings enabled developers and customer representatives discussing requirements in detail before and/or during development.

Although it is not visible in the workspace layout figure (Figure 1), each team was constructed using the same template – in addition to the usual roles of senior developer and junior developer and ScrumMaster, each team included the roles test and functional responsible. The functional responsible was collaborating closely with the product owner. For Alfa and Beta teams these roles were filled by people from Alfa and Beta, respectively. For the Gamma teams, people from Gamma's business side, primarily casework executives or domain resources from Gamma, filled these roles. This arrangement was carefully planned and succeeded in ensuring that ‘the voice’ of the product owner was present in each construction team. Hence, questions regarding issues from ‘problem world’ could be resolved locally in the construction team. This helps improve scalability.

3.1 The overall development process

Omega was a very large undertaking, and from the outset it was structured to simplify management and organization. Most importantly, Omega had 12 releases with the overall structure as shown in figure 2.

Each release was bootstrapped with a phase called *needs analysis* to define the high level requirements of the system. During this phase, Gamma, primarily on their own, identified and described a set of high-level user stories named *epics* that the system should accommodate. The epics gave direction and substance to the subsequent solution description phase. Not only did the epics enable rough estimates and prioritization, but they also contained the initial ideas for what the final system should do. The *solution description* phase contained the bulk part of requirements work. During solution description, high-level requirements from the epics were broken down into more detailed user stories, with sufficient detail so that construction teams could implement them. Finally, in the *construction* phase, user stories were reviewed and ‘signed off’ by construction teams. This included adding final

Needs analysis

Solution description

Construction (5-7 iterations)

Figure 2. The development phases in Omega.

acceptance criteria and [story points] estimates. User stories were then implemented.

3.2 Bridging of problem- and solution-worlds in conversations

An interesting aspect of the Omega project was the overall character of the requirement process. Rather than naming it ‘problem description’ or ‘system requirements’ or some such, the Omega project decided to adopt a hybrid model in which it was named it ‘solution description.’ We believe that this naming was an important enabler for a more effective bridging of problem- and solution-worlds.

Solution description was a collaborative activity in which participants from Gamma, Alfa and Beta worked together. The work was organized in dedicated solution description teams. Representatives from the problem-domain discussed possible solutions with the representatives from the solution-domain. In this joint activity, Omega could bridge the typical barrier between requirements engineering and software design.

During solution description, user stories were expanded to include more details such as design and architectural decisions. Both textual and figurative artifacts were produced, e.g. domain models and any forms of sketches. One respondent explained: “We had a drawing of our system components where it was indicated where a particular release was expected to make an impact. For each of those areas, it was described – using keywords – what those impacts would mean in terms of changes.” Hence, solution description was able to accomplish important work in the solution-world by identifying many dependencies in the system.

Solution description work had a temporal shift in character. One respondent explained: “We called it functional solution description, and it progressed towards a technical solution description. [...] In the beginning it was functional resources working on it, but gradually taken over by technical resources. It was optimal.”

Gamma collaborated with the two other contractors to produce these solution descriptions. Omega's development manager named this “customer-driven solution description.”

Furthermore, the solution description process was gradually changed from an activity only done before the construction phase (as illustrated in figure 3) to a process named continuous solution description. Now, solution description was done prior to each of the iterations within a construction phase. The boundaries between solution description and actual construction were [yet again] pierced by sporadic and pragmatic collaborations across solution description teams and construction teams as solution description teams found it practical to confer with construction teams for clarifications and confirmations of issues.

Thus, at different levels, solution description in Omega took the form of a conversation that contributed to bring requirements work into the solution space and thereby narrowed the before mentioned gap.

3.3 Effective RE conversations

A conversation has a fundamental purpose; to establish shared understanding through dialogue. In Omega, we learned that as parties' understanding of the problem domain increased, the effectiveness of communication increased as well. One respondent explained: “It changed over time, you understand the domain better, the teams understand the domain. The teams eventually know more about the details that those who try to describe the solution. So, from describing a bit too much – and there would always be meetings to clarify – one could describe much less. And the teams sorted out the details.”

Furthermore, conversations across boundaries were established as a productive approach to resolve issues – both in terms of understanding the problem – and understanding the solution. One respondent from Beta explained: “It was always room for the business side to approach us to discuss the solution. Our developers would never discourage that and ‘hide’ behind the target-price contract regime. It was never a commercial conflict in discussing the system – it was always ok to do that.”

In addition, partly due to the extensive use of demos of prototypes, the whole conversation that merged problem with solution was kept on track in each iteration.

Conversations in action-oriented organizations such as Omega must be guided by a shared purpose so as to avoid the trap of endless political debate and ‘discussing anything.’ Conversations in Omega were motivated by a strong and widely shared notion of purpose: To complete the system on time. Sustaining effective conversations throughout the project is vital. However, conversations cannot be effective unless there is considerable agreement and ‘unity’ of fundamental values and objectives. We learned from our respondents that they perceived Omega to succeed very well in establishing a common goal.

Furthermore, we should note that requirements engineering in Omega also involved solitary work. The idea of the conversation is not to suggest that all epistemic work is done in discussions or

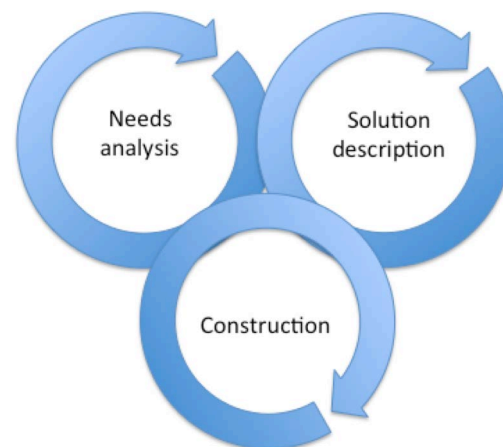


Figure 3. The conversational model of development in Omega

meetings.

Omega was delivered on time, and within the budget. Hence, it seems fair to suggest that the conversations between Gamma, Alfa and Beta in the solution description phase both supported productive collaboration and produced solution descriptions that were effective for guiding the actual construction work.

4. CONCLUSIONS

To our knowledge, this is the first attempt to use the conversation model of software development to understand requirements engineering in agile projects.

Our findings suggest that there were three principal enabling factors for the conversation:

1. The work space layout and the table seating was specifically designed to promote effective communication;
2. Continuous and overlapping processes with a combination of formal and informal meetings;
3. Project culture and project management that promoted face-to-face communication and that rejected written reports.

The conversation model of software development gives us a richer understanding of the complexities that requirements engineering in agile software development must deal with, but it also allows us to understand better the processes that should be supported in order to accommodate productive collaboration between the representatives of the problem-world and those representing the solution-world.

Although this case may seem extraordinary in its approach to open and conversation-oriented cooperation, we should remember that the Scandinavian work tradition has been a great enabler for team-based and participative work since the 1970-ies in Norway [13]. Some of the credit for making Omega possible should therefore go to those that established this fundament.

5. ACKNOWLEDGMENTS

The work on this article was supported by the SINTEF internal project "Agile project management in large development projects" as well as the research project "Agile 2.0" funded by the Norwegian Research Council under grant no. 236759.

6. REFERENCES

- [1] D. B. Walz, J. J. Elam, and B. Curtis, "Inside a software-design team: Knowledge acquisition, sharing and integration," *Communications of the ACM*, vol. 36, pp. 63-77, Oct 1993.
- [2] A. Mackenzie and S. Monk, "From cards to code: How extreme programming re-embodies programming as a collective practice," *Computer Supported Cooperative Work*, vol. 13, pp. 91-117, 2004.
- [3] P. N. Robillard, "The role of knowledge in software development," *Communications of the ACM*, vol. 42, pp. 87-92, 1999.
- [4] D. Damian and J. Chisan, "An empirical study of the complex relationships between requirements engineering processes and other processes that lead to payoffs in productivity, quality, and risk management," *IEEE Transactions on Software Engineering*, vol. 32, pp. 433-453, Jul 2006.
- [5] B. H. C. Cheng and J. M. Atlee, "Current and Future Research Directions in Requirements Engineering," in *Design Requirements Engineering: A Ten-Year Perspective*, vol. 14, K. Lyytinen, P. Loucopoulos, J. Mylopoulos, and B. Robinson, Eds., ed Berlin: Springer-Verlag Berlin, 2009, pp. 11-43.
- [6] B. Nuseibeh, "Weaving together requirements and architectures," *IEEE Computer*, vol. 34, pp. 115-117, 2001.
- [7] S. Nerur and V. Balijepally, "Theoretical reflections on agile development methodologies: The traditional goal of optimization and control is making way for learning and innovation," *Communications of the ACM*, vol. 50, pp. 79-83, Mar 2007.
- [8] B. Fitzgerald, G. Hartnett, and K. Conboy, "Customising agile methods to software practices at Intel Shannon," *European Journal of Information Systems*, vol. 15, pp. 200-213, 2006.
- [9] B. Ramesh, L. Cao, and R. Baskerville, "Agile requirements engineering practices and challenges: an empirical study," *Information Systems Journal*, vol. 20, pp. 449-480, Sep 2010.
- [10] T. Bratteteig, "Making change: Dealing with relations between design and use," PhD thesis, Department of Informatics, University of Oslo, Oslo, Norway, 2003.
- [11] S. D. N. Cook and J. S. Brown, "Bridging epistemologies: The generative dance between organizational knowledge and organizational knowing," *Organization Science*, vol. 10, pp. 381-400, Jul-Aug 1999.
- [12] J. Saldaña, *The coding manual for qualitative researchers*, Second ed.: Sage Publications Ltd, 2009.
- [13] F. Emery and E. Thorsrud, *Democracy at work*. Leiden, Netherlands: Martinus Nijhoff, 1976.